

customer-churn-analysis

December 6, 2025

```
[3]: import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
df= pd.read_csv('Customer Churn.csv')
```

```
[4]: df
```

[4]:	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	\
0	7590-VHVEG	Female	0	Yes	No	1	
1	5575-GNVDE	Male	0	No	No	34	
2	3668-QPYBK	Male	0	No	No	2	
3	7795-CFOCW	Male	0	No	No	45	
4	9237-HQITU	Female	0	No	No	2	
...	...	...	...	...	...	...	
7038	6840-RESVB	Male	0	Yes	Yes	24	
7039	2234-XADUH	Female	0	Yes	Yes	72	
7040	4801-JZAZL	Female	0	Yes	Yes	11	
7041	8361-LTMKD	Male	1	Yes	No	4	
7042	3186-AJIEK	Male	0	No	No	66	
	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	\	
0	No	No phone service	DSL	No	...		
1	Yes	No	DSL	Yes	...		
2	Yes	No	DSL	Yes	...		
3	No	No phone service	DSL	Yes	...		
4	Yes	No	Fiber optic	No	...		
...	...	...	...	...	...		
7038	Yes	Yes	DSL	Yes	...		
7039	Yes	Yes	Fiber optic	No	...		
7040	No	No phone service	DSL	Yes	...		
7041	Yes	Yes	Fiber optic	No	...		
7042	Yes	No	Fiber optic	Yes	...		
	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	\	
0	No	No	No	No	Month-to-month		
1	Yes	No	No	No	One year		

2	No	No	No	No	Month-to-month
3	Yes	Yes	No	No	One year
4	No	No	No	No	Month-to-month
...	...	...	...	...	...
7038	Yes	Yes	Yes	Yes	One year
7039	Yes	No	Yes	Yes	One year
7040	No	No	No	No	Month-to-month
7041	No	No	No	No	Month-to-month
7042	Yes	Yes	Yes	Yes	Two year

	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	\
0	Yes	Electronic check	29.85	29.85	
1	No	Mailed check	56.95	1889.5	
2	Yes	Mailed check	53.85	108.15	
3	No	Bank transfer (automatic)	42.30	1840.75	
4	Yes	Electronic check	70.70	151.65	
...	...	...	...	...	
7038	Yes	Mailed check	84.80	1990.5	
7039	Yes	Credit card (automatic)	103.20	7362.9	
7040	Yes	Electronic check	29.60	346.45	
7041	Yes	Mailed check	74.40	306.6	
7042	Yes	Bank transfer (automatic)	105.65	6844.5	

Churn	
0	No
1	No
2	Yes
3	No
4	Yes
...	...
7038	No
7039	No
7040	No
7041	Yes
7042	No

[7043 rows x 21 columns]

```
[5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerID      7043 non-null   object
1   gender          7043 non-null   object
```

```

2 SeniorCitizen      7043 non-null  int64
3 Partner            7043 non-null  object
4 Dependents         7043 non-null  object
5 tenure             7043 non-null  int64
6 PhoneService       7043 non-null  object
7 MultipleLines      7043 non-null  object
8 InternetService    7043 non-null  object
9 OnlineSecurity     7043 non-null  object
10 OnlineBackup       7043 non-null  object
11 DeviceProtection  7043 non-null  object
12 TechSupport       7043 non-null  object
13 StreamingTV       7043 non-null  object
14 StreamingMovies   7043 non-null  object
15 Contract          7043 non-null  object
16 PaperlessBilling  7043 non-null  object
17 PaymentMethod     7043 non-null  object
18 MonthlyCharges    7043 non-null  float64
19 TotalCharges      7043 non-null  object
20 Churn             7043 non-null  object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

```

1 in total charges there is space for 0rs

2 so we convert that space in 0

```

[6]: df["TotalCharges"] = df["TotalCharges"].replace(" ", "0")
     df["TotalCharges"] = df["TotalCharges"].astype("float")

```

```

[7]: df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null  object
1   gender                7043 non-null  object
2   SeniorCitizen         7043 non-null  int64
3   Partner               7043 non-null  object
4   Dependents            7043 non-null  object
5   tenure                7043 non-null  int64
6   PhoneService          7043 non-null  object
7   MultipleLines         7043 non-null  object
8   InternetService       7043 non-null  object
9   OnlineSecurity        7043 non-null  object
10  OnlineBackup          7043 non-null  object

```

```

11 DeviceProtection 7043 non-null object
12 TechSupport      7043 non-null object
13 StreamingTV      7043 non-null object
14 StreamingMovies  7043 non-null object
15 Contract         7043 non-null object
16 PaperlessBilling 7043 non-null object
17 PaymentMethod    7043 non-null object
18 MonthlyCharges   7043 non-null float64
19 TotalCharges     7043 non-null float64
20 Churn            7043 non-null object
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB

```

3 check no null value is sum is =0 than no null value

```
[8]: df.isnull().sum().sum().sum()
```

```
[8]: np.int64(0)
```

```
[9]: df.describe()
```

```

[9]:      SeniorCitizen      tenure  MonthlyCharges  TotalCharges
count      7043.000000  7043.000000      7043.000000    7043.000000
mean         0.162147    32.371149         64.761692    2279.734304
std          0.368612    24.559481         30.090047    2266.794470
min           0.000000     0.000000         18.250000     0.000000
25%           0.000000     9.000000         35.500000    398.550000
50%           0.000000    29.000000         70.350000   1394.550000
75%           0.000000    55.000000         89.850000   3786.600000
max           1.000000    72.000000        118.750000   8684.800000

```

4 check duplicate in data and any double entry

```
[10]: df.duplicated().sum()
```

```
[10]: np.int64(0)
```

```
[11]: df["customerID"].duplicated().sum()
```

```
[11]: np.int64(0)
```

5 in SeniorCitizen colume 0=yes ,1 =No so we make it

```
[12]: def con(val):
      if val==1:
          return "yes"
      else:
          return "No"
      df['SeniorCitizen']=df['SeniorCitizen'].apply(con)
```

```
[13]: df
```

```
[13]:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	\
0	7590-VHVEG	Female	No	Yes	No	1	
1	5575-GNVDE	Male	No	No	No	34	
2	3668-QPYBK	Male	No	No	No	2	
3	7795-CFOCW	Male	No	No	No	45	
4	9237-HQITU	Female	No	No	No	2	
...	...	...	...	...	...	...	
7038	6840-RESVB	Male	No	Yes	Yes	24	
7039	2234-XADUH	Female	No	Yes	Yes	72	
7040	4801-JZAZL	Female	No	Yes	Yes	11	
7041	8361-LTMKD	Male	yes	Yes	No	4	
7042	3186-AJIEK	Male	No	No	No	66	

	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	\
0	No	No phone service	DSL	No	...	
1	Yes	No	DSL	Yes	...	
2	Yes	No	DSL	Yes	...	
3	No	No phone service	DSL	Yes	...	
4	Yes	No	Fiber optic	No	...	
...	...	...	...	...	...	
7038	Yes	Yes	DSL	Yes	...	
7039	Yes	Yes	Fiber optic	No	...	
7040	No	No phone service	DSL	Yes	...	
7041	Yes	Yes	Fiber optic	No	...	
7042	Yes	No	Fiber optic	Yes	...	

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	\
0	No	No	No	No	Month-to-month	
1	Yes	No	No	No	One year	
2	No	No	No	No	Month-to-month	
3	Yes	Yes	No	No	One year	
4	No	No	No	No	Month-to-month	
...	...	...	...	...	...	
7038	Yes	Yes	Yes	Yes	One year	
7039	Yes	No	Yes	Yes	One year	
7040	No	No	No	No	Month-to-month	

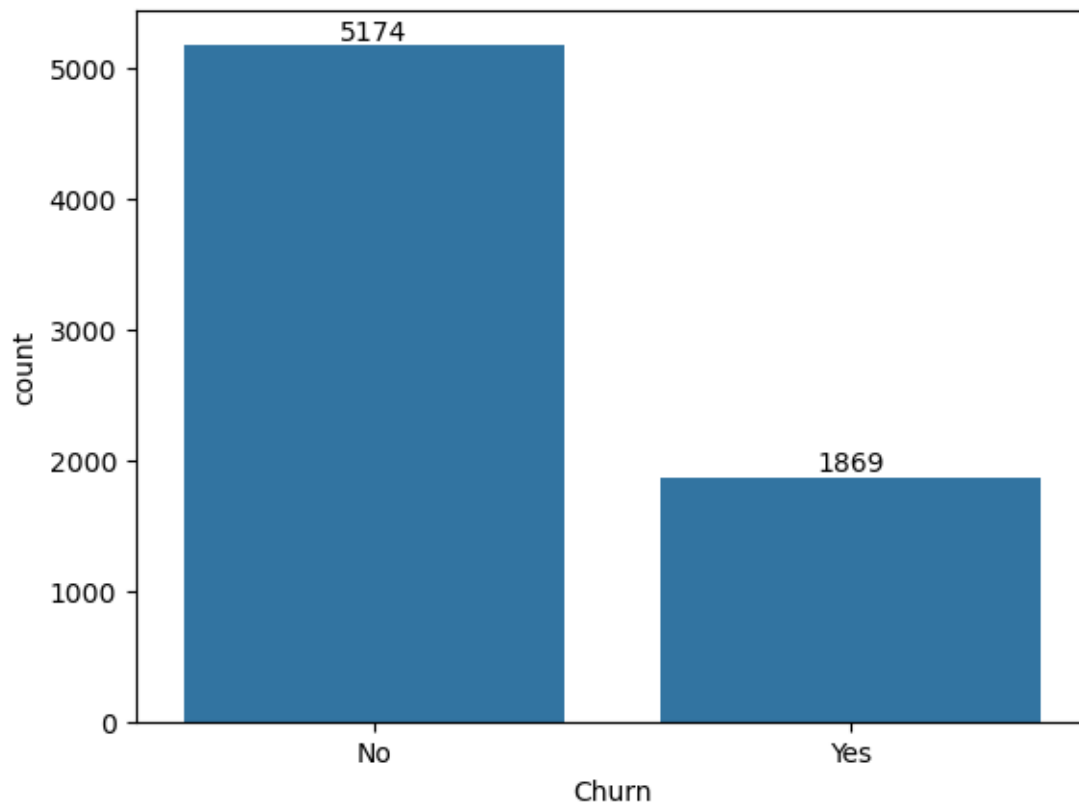
7041	No	No	No	No	Month-to-month
7042	Yes	Yes	Yes	Yes	Two year

	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	\
0	Yes	Electronic check	29.85	29.85	
1	No	Mailed check	56.95	1889.50	
2	Yes	Mailed check	53.85	108.15	
3	No	Bank transfer (automatic)	42.30	1840.75	
4	Yes	Electronic check	70.70	151.65	
...	...	...	...	...	
7038	Yes	Mailed check	84.80	1990.50	
7039	Yes	Credit card (automatic)	103.20	7362.90	
7040	Yes	Electronic check	29.60	346.45	
7041	Yes	Mailed check	74.40	306.60	
7042	Yes	Bank transfer (automatic)	105.65	6844.50	

	Churn
0	No
1	No
2	Yes
3	No
4	Yes
...	...
7038	No
7039	No
7040	No
7041	Yes
7042	No

[7043 rows x 21 columns]

```
[18]: ax=sns.countplot(x='Churn',data=df)
ax.bar_label(ax.containers[0])
plt.show()
```



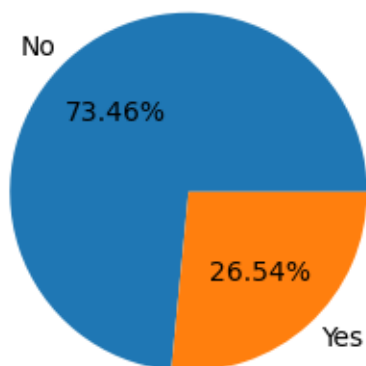
6 group by karna

```
[20]: gb = df["Churn"].value_counts()  
gb
```

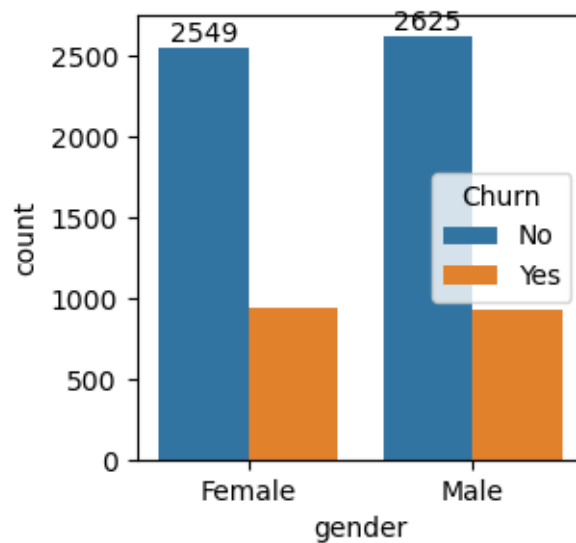
```
[20]: Churn  
No      5174  
Yes      1869  
Name: count, dtype: int64
```

```
[27]: plt.figure(figsize=(3,4)) # arjust size  
gb = df["Churn"].value_counts()  
  
plt.pie(gb, labels=gb.index, autopct="%1.2f%%")  
plt.title("Churn Distribution")  
plt.show()
```

Churn Distribution

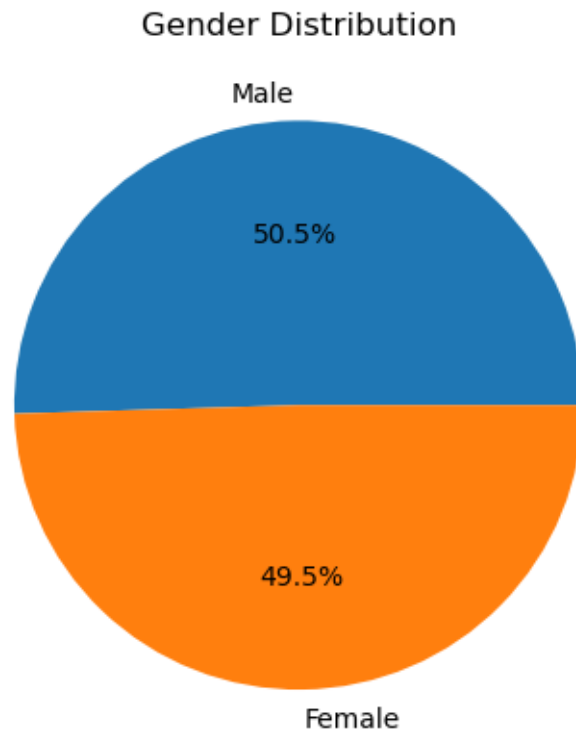


```
[49]: # male vs female
plt.figure(figsize=(4,4))
ax=sns.countplot(x='gender',data=df,hue="Churn")
ax.bar_label(ax.containers[0])
plt.show()
```

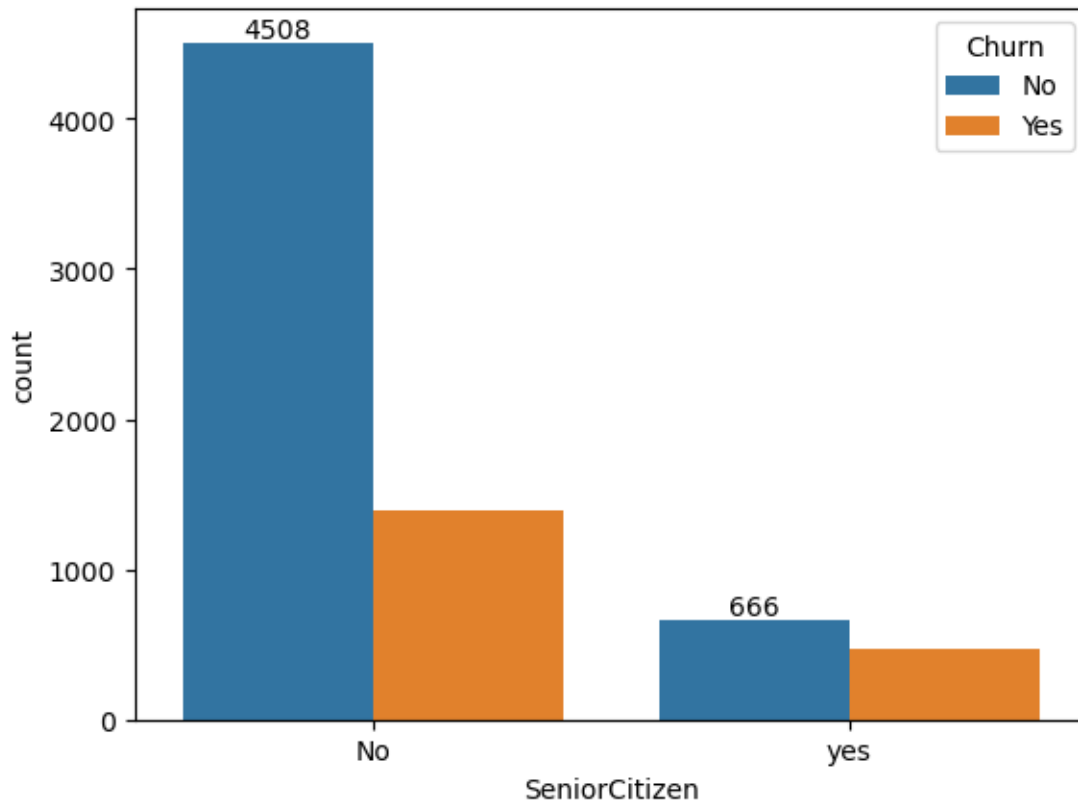


```
[35]: gb = df["gender"].value_counts() # count each gender

plt.pie(gb, labels=gb.index, autopct="%1.1f%%")
plt.title("Gender Distribution")
plt.show()
```



```
[52]: ax=sns.countplot(x='SeniorCitizen',data=df,hue="Churn")  
ax.bar_label(ax.containers[0])  
plt.show()
```



```
[54]: # groupby to get counts
group = df.groupby(['SeniorCitizen', 'Churn']).size().unstack()

# convert to percentage
percent = group.div(group.sum(axis=1), axis=0) * 100

plt.figure(figsize=(6,4))

# bottom part (Churn = No)
plt.bar(percent.index, percent['No'], label="Churn = No")

# top part (Churn = Yes)
plt.bar(percent.index, percent['Yes'], bottom=percent['No'], label="Churn = Yes")

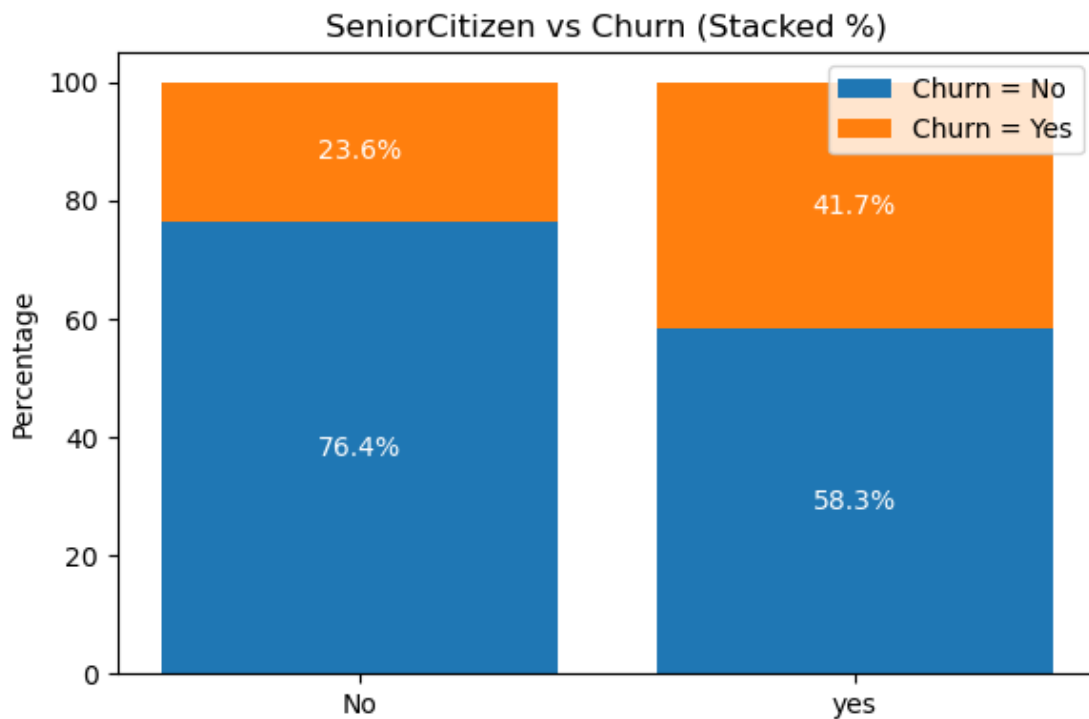
# add percentage labels
for i, idx in enumerate(percent.index):
    # label for 'No'
    plt.text(i, percent['No'][idx]/2,
             f"{percent['No'][idx]:.1f}%",
             ha='center', va='center', color='white', fontsize=10)
```

```

# label for 'Yes'
plt.text(i, percent['No'][idx] + percent['Yes'][idx]/2,
        f"{percent['Yes'][idx]:.1f}%",
        ha='center', va='center', color='white', fontsize=10)

plt.ylabel("Percentage")
plt.title("SeniorCitizen vs Churn (Stacked %)")
plt.legend()
plt.tight_layout()
plt.show()

```

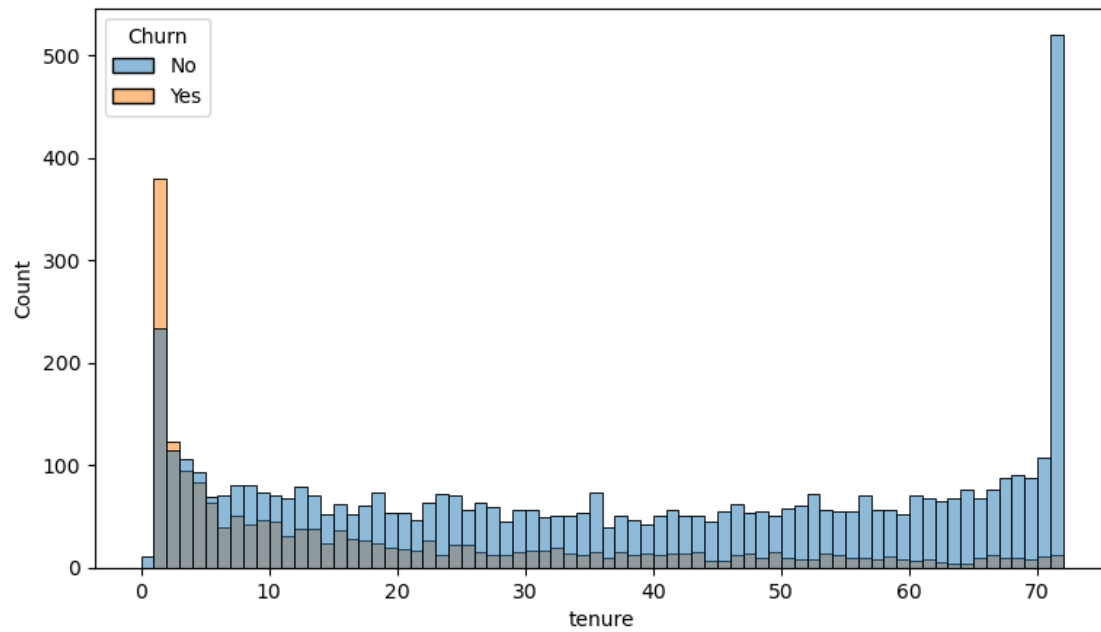


#compair a greater percenatge of people in senior citizen category have churned

```

[59]: plt.figure(figsize=(9,5))
sns.histplot(x='tenure',data=df,bins=72,hue="Churn")
plt.show()

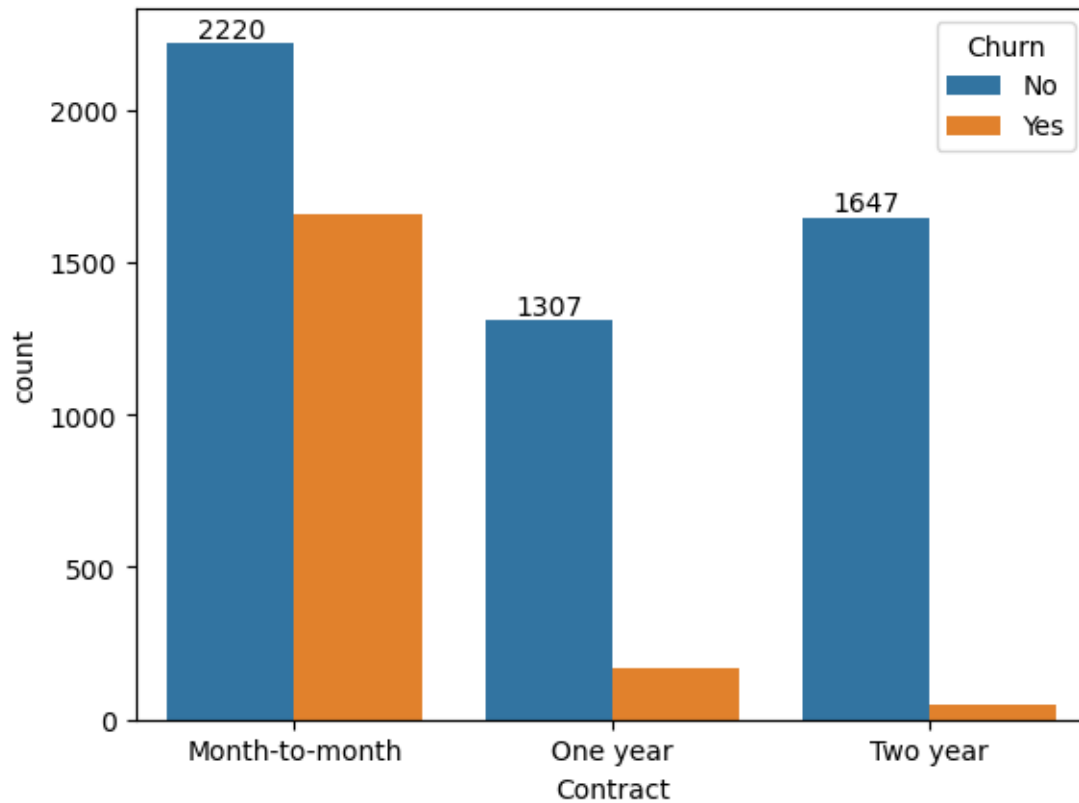
```



#people who have use more than 2-3 month that they with us

```
[61]: ax=sns.countplot(x="Contract",data=df,hue="Churn")
      ax.bar_label(ax.containers[0])
```

```
[61]: [Text(0, 0, '2220'), Text(0, 0, '1307'), Text(0, 0, '1647')]
```



#who have month to month that likely churn and more than with us

```
[66]: import matplotlib.pyplot as plt
import seaborn as sns

# columns you want to plot
cols = [
    "PhoneService", "MultipleLines", "InternetService",
    "OnlineSecurity", "OnlineBackup", "DeviceProtection",
    "TechSupport", "StreamingTV", "StreamingMovies"
]

plt.figure(figsize=(15,12))          # big figure for 9 plots

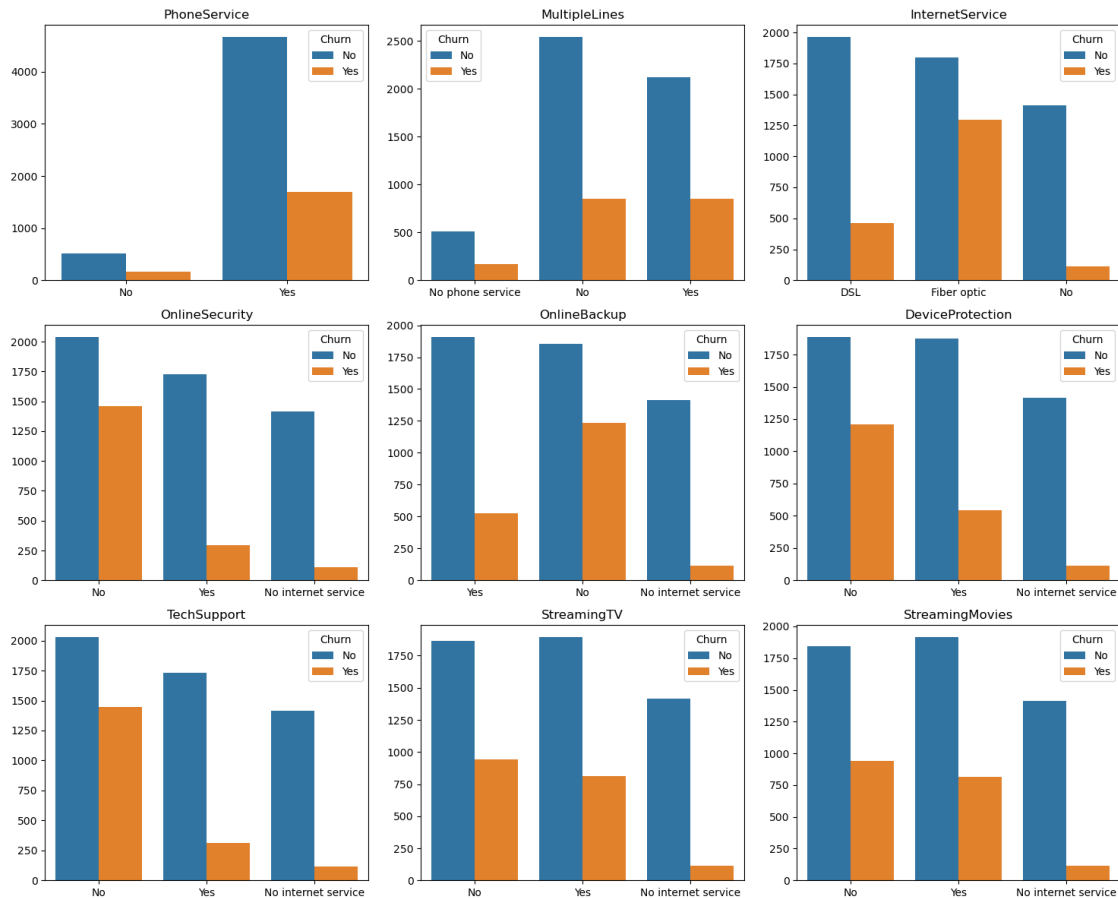
# loop through columns with index
for i, col in enumerate(cols, 1):

    plt.subplot(3, 3, i)              # create 3 rows, 3 columns
    sns.countplot(x=col, data=df, hue="Churn") # countplot

    plt.title(col)                    # title
```

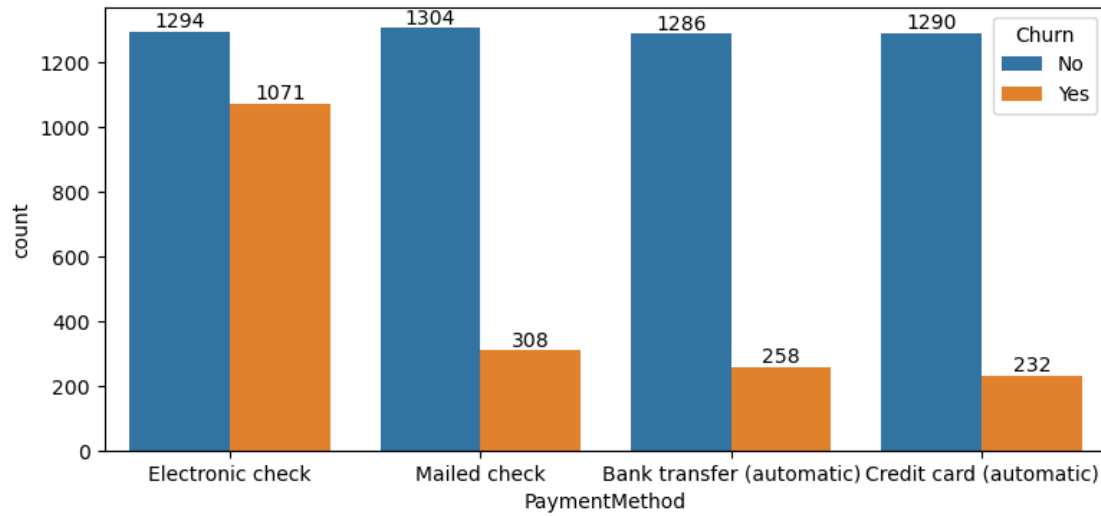
```
plt.xlabel("") # hide x-label (clean look)
plt.ylabel("") # hide y-label
plt.xticks(rotation=20) # angle for readability

plt.tight_layout() # adjust spacing
plt.show() # show all 9 together
```



#Most customers who did not churn appear in the “No” category for almost every service feature like OnlineSecurity, OnlineBackup, TechSupport, and DeviceProtection. #Churn is noticeably higher among customers who lack these services (especially “No internet service” and “No protection” categories). #Customers who have internet-related services show more churn compared to phone-based services. #Overall, missing security and support services strongly relate to higher churn.

```
[70]: plt.figure(figsize=(9,4))
ax=sns.countplot(x='PaymentMethod',data=df,hue="Churn")
ax.bar_label(ax.containers[0])
ax.bar_label(ax.containers[1])
plt.show()
```



#Here are the short, clear points: #1. Churn is **highest** among customers using **Electronic Check**, showing weaker retention in this payment method. #2. Customers paying through **Mailed Check, Bank Transfer, and Credit Card (automatic)** churn far less, suggesting **automatic or stable #payment methods** are linked to better customer retention.

[]: